

One-Page Summary Report

1. Document Ingestion & Knowledge Pyramid Approach

The system implements a modular document ingestion pipeline using a sliding window strategy and hierarchical knowledge representation.

A **character-based sliding window (2000–3000 characters)** with overlap is used to split large documents into smaller chunks while preserving context across boundaries.

Each chunk is processed into a **4-layer Knowledge Pyramid**:

- **Raw Text:** Original extracted content
- **Chunk Summary:** Lightweight summarization using first few sentences
- **Category/Theme:** Rule-based keyword extraction for classification
- **Distilled Knowledge:** Dense vector embeddings generated using Sentence Transformers

This layered abstraction allows efficient retrieval at multiple semantic levels.

2. Retrieval Strategy

User queries are processed using semantic similarity search:

- Queries are converted into embeddings using all-MiniLM-L6-v2
- FAISS is used for efficient nearest neighbor search
- Top-K relevant chunks are retrieved
- Multi-level knowledge improves robustness and relevance

The system follows a **RAG-inspired architecture**, enabling scalable and context-aware information retrieval.

3. GSM8K Training Setup

The GSM8K dataset is loaded from Hugging Face and split into:

- **Training:** 3000 samples
- **Evaluation:** 1000 samples

A lightweight causal language model (simulating LLaMA 3.2 1B) is fine-tuned using **LoRA (Low-Rank Adaptation)** for memory efficiency.

The training pipeline includes:

- Tokenization of problem-answer pairs
- Supervised fine-tuning (SFT) with step-by-step reasoning
- Training loop with loss computation and optimizer updates
- Evaluation using Exact Match accuracy

4. Key Design Decisions

- **Character-based chunking** chosen for simplicity and speed
- **Lightweight summarization** used to reduce computational overhead
- **FAISS indexing** for scalable similarity search
- **LoRA fine-tuning** to enable efficient training on limited hardware
- **Lazy model loading** to reduce API startup latency
- **Modular architecture** separating ingestion, retrieval, and routing

5. Bonus: Reasoning-Aware Adapter (Concept)

A routing layer is designed to classify queries and activate specialized modules:

- **Math queries** → step-by-step reasoning module
- **Legal queries** → structured retrieval
- **General queries** → semantic search

This is implemented using a rule-based routing function and can be extended using a classifier.

Conclusion

This project demonstrates a scalable and modular approach to building AI-driven retrieval systems. By combining hierarchical knowledge representation, semantic embeddings, and efficient indexing, the system enables accurate and context-aware query handling while maintaining computational efficiency.